# Formative 2: Modeling Human Activity States Using Hidden Markov Models

Kayumba David Pontient, African Leadership University. Individual submission.
GitHub repository: https://github.com/DavidPontient/Hidden-Markov-Models

## 1. Background and motivation

Human activity recognition turns the raw motion signals from a phone into a simple answer about what a person is doing. It is used in health monitoring, fall detection and fitness tracking. My use case comes from my own project, a platform that makes land ownership and home construction in Rwanda more transparent. An owner who is far from a building site finds it hard to know how much real work is happening. A phone worn on a worker's belt records motion, and a model that reads that motion can tell active work, such as walking or jumping down from a height, from standing around or from a phone and tools left still. The true activity is hidden behind noisy sensor readings, and activities follow each other in time, so a Hidden Markov Model is a good fit. In this project I recorded my own accelerometer and gyroscope data for four activities (standing, walking, jumping and still), extracted time and frequency features, and built an HMM from scratch with numpy to decode the activity from those features.

## 2. Data collection and pre-processing

I recorded with the Sensor Logger app on my iPhone 13, with only the accelerometer and the gyroscope turned on. For standing I held the phone steady at waist level, for walking I kept an even pace, for jumping I jumped continuously, and for still I left the phone flat on a table. I made 50 training recordings of about 8 to 9 seconds each: 12 standing, 12 jumping, 13 walking and 13 still. I cut 0.5 seconds from the start and the end of every file, because that is where I pressed start and moved the phone. After this cleaning, each activity has between 1.65 and 1.77 minutes of data, which is above the 1 minute 30 second minimum. Every file is saved as a CSV with a timestamp column, the three accelerometer axes, the three gyroscope axes and the activity label, in the folder data/train/<activity>. The two test recordings are in data/test.

| Member | Phone | Original sampling rate | Rate used for the model |
|---|---|---|---|
| David Pontient | iPhone 13 | 99.58 Hz on average (99.56 to 99.59 Hz) | 100 Hz for every file |

*Table 1. Sampling rate of each group member. I am the only member, so all 52 recordings come from one phone.*

Sensor Logger reports about 99.6 Hz and not exactly 100 Hz. I merged the two sensors on their shared time range and used linear interpolation to put both on one regular 100 Hz grid. I chose linear interpolation because the signals change smoothly at this rate. The same function would bring phones with different rates to 100 Hz, so the method would also work for a team.

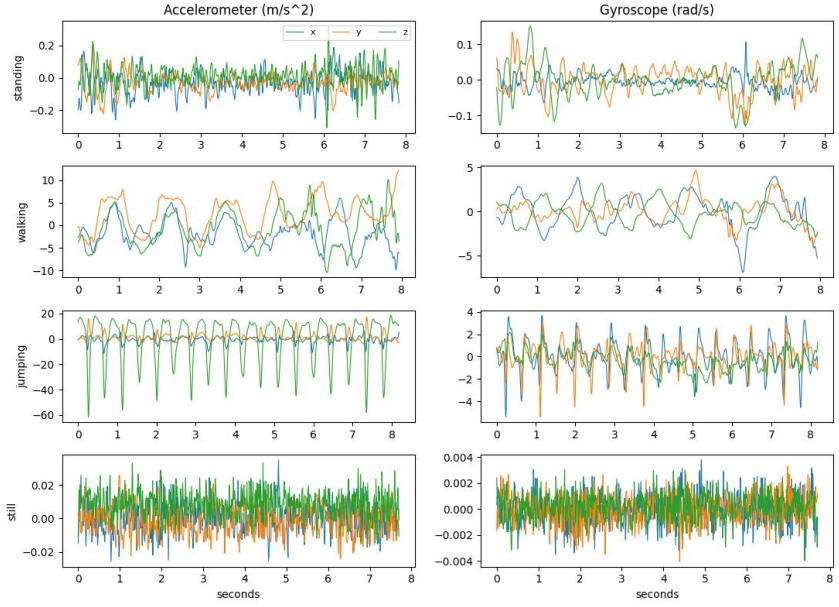



*Figure 1. One cleaned training file for each activity. Jumping gives large repeated spikes, walking gives a smooth swing, standing gives small random movement, and still gives only sensor noise (note the very different y axis scales).*

## Window size and features

I cut each recording into windows of 2 seconds, which is 200 samples at 100 Hz, with 50 percent overlap, so a new window starts every second. I chose 2 seconds because a normal walking pace is about 2 steps per second, so a window holds several full steps and the FFT can see the rhythm. The FFT frequency resolution is 1 divided by the window length, which gives 0.5 Hz steps. A much shorter window would blur the difference between walking and jumping, and a much longer one would mix two activities when I change from one to the other. This gave 333 training windows (85 jumping, 79 standing, 82 still and 87 walking). I computed 8 features for the accelerometer and the same 8 for the gyroscope, so 16 features in total.

| Domain | Feature | Why it helps to tell the activities apart |
|---|---|---|
| Time | mean of magnitude | Overall size of the motion. |
| Time | standard deviation | Tiny when still or standing, large for walking and jumping. |
| Time | RMS | Energy of the motion, close to the standard deviation but also reflects the mean. |
| Time | signal magnitude area (SMA) | Total movement over all three axes, mean of \|x\|+\|y\|+\|z\|. |
| Time | correlation between axes | Jumping moves along one direction, so axes are linked. Noise gives weak links. |
| Frequency | dominant frequency (FFT) | Repeat rate of the movement, searched between 0.5 and 20 Hz. |
| Frequency | spectral energy (FFT) | Strength of the movement based on its frequency content. |
| Frequency | spectral entropy (FFT) | Repeated movement puts power in a few frequencies (low entropy), noise spreads it (high entropy). |

*Table 2. The 8 features computed for each sensor in every window.*

Some features are about 0.01 when the phone is still and about 8 when I am jumping, and a Gaussian cannot describe a gap this big. So I took the log of the features that measure the size of the motion (mean, standard deviation, RMS, SMA and spectral energy) and then applied z-score normalization to all 16 features. I chose z-score because the Gaussian emissions work best when all features are on a similar scale, and it does not squash the data into a fixed range the way min-max scaling does. The mean and standard deviation come from the training windows only, and the same numbers were applied to the test windows.

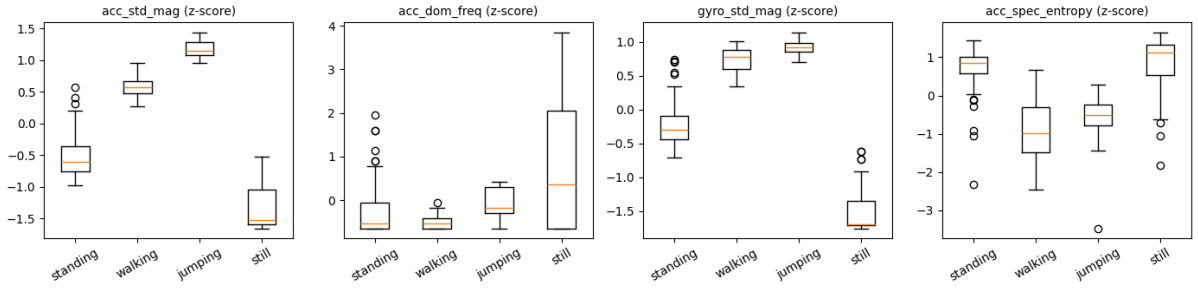



*Figure 2. Four normalized features by activity for the training windows. The standard deviation features separate the four activities well, while the dominant frequency overlaps a lot.*

## Video evidence of data collection

Each video is 5 seconds long and shows my full body and my face, with good light, while I hold the phone and record the activity.

| Activity | Video link |
|---|---|
| Standing | https://youtube.com/shorts/VZj02NgXXms?feature=share |
| Walking | https://youtube.com/shorts/1XH1wIZqcSI?feature=share |
| Jumping | https://youtube.com/shorts/maRzpEFqmYc?feature=share |
| Still (no movement) | https://youtube.com/shorts/9UcsPFohE3Y?feature=share |

*Table 3. Links to the four data collection videos.*

## 3. HMM setup and implementation details

| Element | My definition |
|---|---|
| Hidden states (Z) | Four activities: standing, walking, jumping, still. |
| Observations (X) | The 16 normalized features of each 2 second window. |
| Initial probabilities (pi) | After training: standing 0.30, walking 0.22, jumping 0.23, still 0.25. |
| Transition matrix (A) | 4 by 4 matrix learned with Baum-Welch (Figure 3). |
| Emission probabilities (B) | One Gaussian per state with a diagonal covariance, so each feature has its own mean and variance in each state. The variance has a floor of 0.02. |

*Table 4. The components of the model.*

Each training file holds one activity, so training on the files one by one would give a transition matrix with almost no switching. To learn transitions I built 60 longer sequences by joining 4 to 6 recordings in a random order, where the next activity is always different from the one before. This gave 1,970 windows. The starting values came from the labeled windows: the mean and variance of each state from the windows of that activity, and the transition and starting counts from the joined sequences. This keeps state 0 as standing, state 1 as walking, state 2 as jumping and state 3 as still while Baum-Welch improves the numbers.

**Algorithms.** Baum-Welch is the right algorithm for training because it learns pi, A and B from unlabeled sequences by maximizing the likelihood of the data. Viterbi is the right algorithm for decoding because it returns the single most likely path of states, and that path is what I compare with the true label of each window to get the confusion matrix, sensitivity, specificity and accuracy. The forward pass gives the log-likelihood that I use for the convergence check. All calculations are done in log space so that long sequences do not underflow.

I wrote a GaussianHMM class with the forward, backward, Viterbi and fit functions. Baum-Welch stops when the improvement in the average log-likelihood per window is smaller than 0.0001, with a limit of 200 iterations as a backup. It converged at iteration 11, and the log-likelihood per window went from -2.9491 at the start to -2.2593 at the end. Viterbi decoding on the joined training sequences gave an accuracy of 0.9269, but this is training data, so it does not show how well the model generalizes.

## 4. Results and interpretation

**Unseen data.** I recorded two new files after the training files: test-01 (44.0 seconds) and test-02 (41.7 seconds). In each file I did the four activities in a row, so the model is tested on whole recordings that it never saw, and also on the switches between activities. I used the same phone and the same person, so this is a new session and not a new person or a new place. I wrote down the activity times while recording and checked them against the movement in the data. In test-01 the times are standing 0 to 10 s, walking 10 to 20 s, jumping 21 to 33 s and still 34 to 43 s. In test-02 they are standing 0 to 10 s, walking 11 to 20 s, jumping 21 to 32 s and still 33 to 41 s. I decoded all 82 windows with Viterbi, but I only scored the 63 windows that sit fully inside one labeled segment (33 in test-01 and 30 in test-02). Windows that touch a gap, where I was changing activity, are not scored because their true label is not clear.

| State (Activity) | Number of Samples | Sensitivity | Specificity | Overall Accuracy |
|---|---|---|---|---|
| Standing | 16 | 1.0000 | 0.9574 | 0.9683 |
| Walking | 15 | 1.0000 | 1.0000 | 1.0000 |
| Jumping | 19 | 1.0000 | 1.0000 | 1.0000 |
| Still | 13 | 0.8462 | 1.0000 | 0.9683 |
| **All activities** | **63** | | | **0.9683** |

*Table 5. Results on the two unseen test files. Sensitivity and specificity are one activity against the rest. The accuracy in each row is for that activity against the rest, and the last row is the accuracy over all four classes (61 of 63 windows correct).*

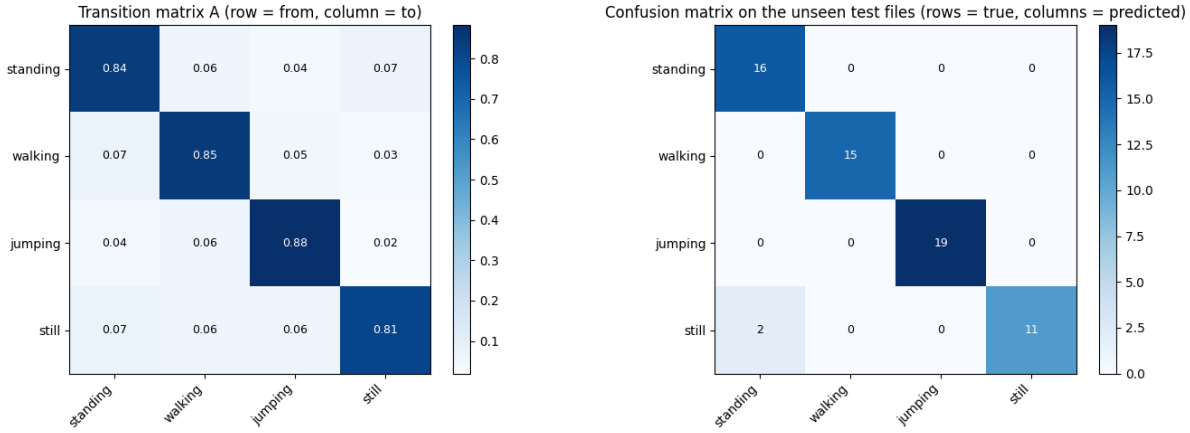



*Figure 3 (left). Transition matrix A, each row is the current activity and each column is the next. Figure 4 (right). Confusion matrix on the unseen test files, rows are the true activity and columns are the prediction.*

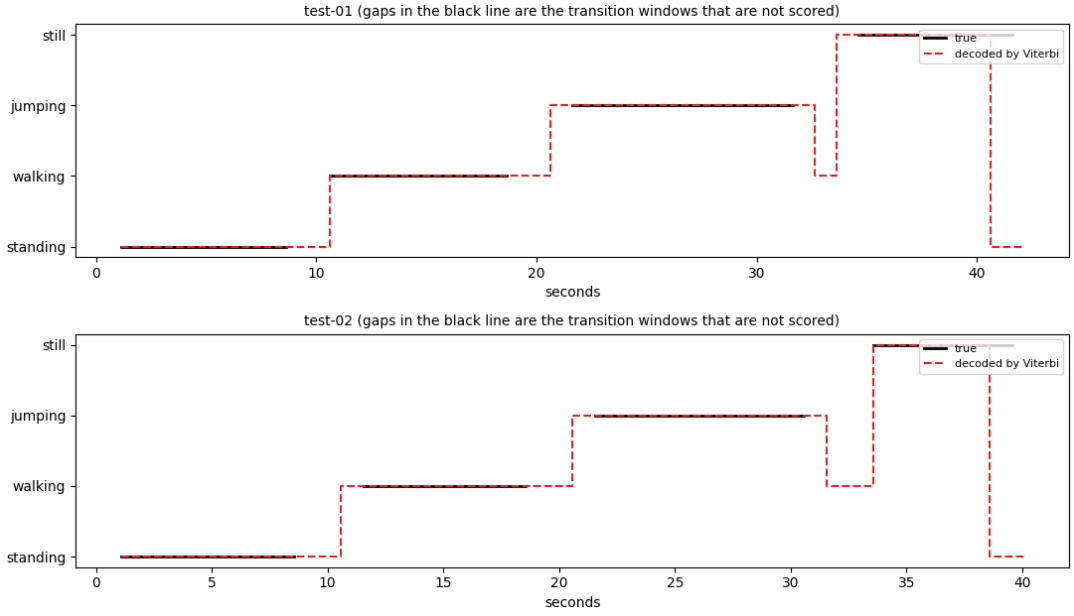



*Figure 5. The true activity (black) and the sequence decoded by Viterbi (red dashed) for the two test files. The decoded line differs from the true line only at the switches and at the very end of each file.*

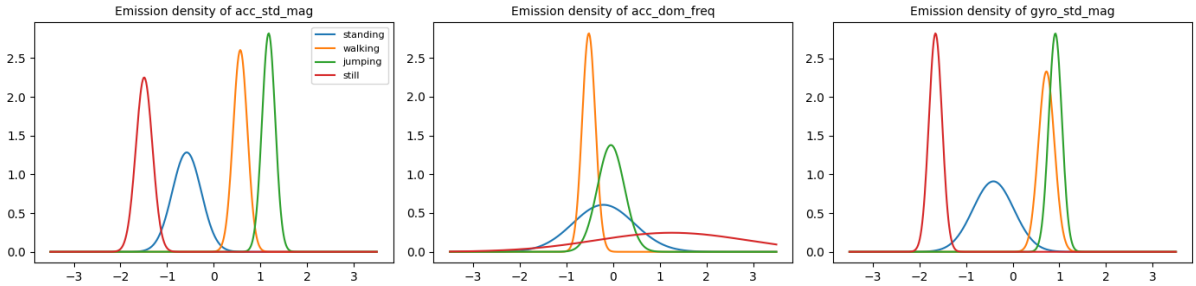



*Figure 6. Emission probabilities (Gaussian densities) of three normalized features in each state. The standard deviation features separate the states clearly, and the dominant frequency overlaps.*

**Does the model generalize?** On these two files, yes. Walking, jumping and standing all had a sensitivity of 1.0, and still had 0.8462. Test-01 reached an accuracy of 0.9697 (32 of 33 windows) and test-02 reached 0.9667 (29 of 30 windows). Both errors are the same kind: the last still window of the file was decoded as standing. At the end of each file I reached for the phone to stop the recording. In test-01 the movement signal rises from about 0.006 to about 0.06 at 41.5 seconds, and test-02 shows small bumps near 39.5 seconds and at the end. So that last window contains some real movement, and standing is an understandable answer. I kept the labels as I first set them and did not change the model to fix these two windows. I should also say that the test data comes from the same phone, the same person and the same day, and it has only 63 scored windows, where one window is worth about 1.6 percentage points. The result shows that the method works on new recordings, but it does not prove that it works for other people or other places.

## 5. Discussion and conclusion

**Easiest and hardest activities.** Jumping was the easiest. It gives huge repeated spikes in the vertical acceleration (down to about -60 m/s2 in Figure 1), so its standard deviation features sit far from the others. Walking was also clear, with sensitivity and specificity of 1.0. The hardest pair was standing and still. When I stand and hold the phone, my body gives small movements of about 0.2 m/s2, and when the phone is flat on a table it only shows sensor noise of about 0.02 m/s2. In my recordings the accelerometer does not include gravity, so both activities read close to zero. The model can separate them only because of the size of this small movement, and any extra movement, like picking up the phone, moves a still window towards standing. This is exactly what the two errors show.

**Transition probabilities.** The diagonal of the transition matrix is between 0.81 and 0.88, and moving to any other activity is between 0.02 and 0.07. This looks realistic, because a person stays in one activity for several seconds before changing. With one window per second, a probability of 0.84 for standing means I stay in it for about 6 windows on average. But I must be honest that these numbers come from how I joined my recordings, which are each 6 or 7 windows long, and not from a real day of behavior. I also forced the next activity to be different, so every switch is almost equally likely. A real building site would show patterns, for example walking between standing and jumping more often than a direct change from still to jumping.

**Noise and sampling rate.** The still recordings show how much noise the sensor makes on its own. The model does not fight this noise, it uses its size as a signal, and the log transform made it possible to put such tiny values and the very large jumping values on one scale. The sampling rate was stable at about 99.6 Hz, which is far above what human movement needs, so the small interpolation to 100 Hz had no visible effect. The dominant frequency was the weakest feature in my data. In Figure 2 and Figure 6 walking and jumping overlap, and for still windows it is random because there is only noise. The energy features did most of the work.

**Improvements.** I would record more data from more people, with the phone in a pocket and on a belt, on different floors and outside, so the model does not depend on one person and one way of holding the phone. I would record long continuous sessions with natural changes between activities, so the transition matrix is learned from real behavior. I would add activities such as sitting and climbing stairs, and other sensors such as the magnetometer or barometer. I would also test other features (for example a band of frequencies between 1 and 4 Hz), a model with more than one Gaussian per state, and a test on a different person.

**Conclusion.** I built an HMM from scratch with Baum-Welch training and Viterbi decoding, and it identified my four activities with an overall accuracy of 0.9683 on two unseen recordings. The method fits my construction site use case, where a phone could show active work and idle time. The result is promising, but it needs data from more people and places before I would trust it on a real site.